

Unit: sendRedirect() vs RequestDispatcher

Servlet and JSP Tutorial

Video Information

Video Title: #9 sendRedirect() method || RequestDispatcher - Include vs Forward || Servlet and JSP Tutorial

Video Link: <https://youtu.be/AUa50Yy8214?si=LwhnNaLif5rXmFdz>

Main Topics Covered:

- `sendRedirect()`
- `RequestDispatcher`
- `forward()`
- `include()`
- Difference between `sendRedirect()` and `RequestDispatcher`
- Passing data using request attributes
- `response.setContentType()`

1. Introduction

In **Servlet and JSP applications**, we often need to move a request from one resource to another.

For example:

- Servlet → JSP
- Servlet → another Servlet
- Servlet → HTML page
- Servlet → another website

Java provides two important mechanisms for this purpose:

1. `sendRedirect()`
2. `RequestDispatcher`

The main difference is:

- `sendRedirect()` tells the browser to make a **new request**.
- `RequestDispatcher` performs the operation **inside the server**.

2. sendRedirect()

Definition

`sendRedirect()` is a method of the `HttpServletResponse` interface that tells the client's browser to go to another URL.

Syntax

```
response.sendRedirect("URL");
```

Example

```
response.sendRedirect("https://www.google.com");
```

In this example, the server tells the browser to request Google.

2.1 Characteristics of sendRedirect()

1. URL Changes

When `sendRedirect()` is used, the URL displayed in the browser changes to the new destination.

2. New Request is Created

The browser receives the redirect instruction and makes a **new HTTP request** to the destination.

Therefore, `sendRedirect()` involves an additional request-response cycle.

3. Can Redirect to an External Website

`sendRedirect()` can be used to send the user to a URL outside the current web application.

Example:

```
response.sendRedirect("https://www.google.com");
```

4. Uses HttpServletResponse

The method is called using the `HttpServletResponse` object.

```
response.sendRedirect("profile.jsp");
```

2.2 How sendRedirect() Works

```
Step 1:
Browser
  |
  | Request
  v
Servlet 1

Step 2:
Servlet 1
  |
  | sendRedirect()
  v
Browser

Step 3:
Browser
  |
  | NEW Request
  v
Servlet 2 / Other URL

Step 4:
Servlet 2
  |
  | Response
  v
Browser
```

Easy Understanding

```
sendRedirect()
  ↓
Server tells Browser to redirect
  ↓
Browser creates a NEW request
  ↓
New resource processes the request
```

2.3 Example Use Cases

`sendRedirect()` can be useful when:

- Redirecting to another website.
- Redirecting to a payment gateway.
- Redirecting the user to another URL.
- Redirecting after a successful operation.

3. RequestDispatcher

Definition

`RequestDispatcher` is an interface used to transfer a request to another resource or include another resource's output in the current response.

It is mainly used for **internal communication between resources** in a web application.

Resources may include:

- Servlets
- JSP pages
- HTML resources

Obtaining RequestDispatcher

```
RequestDispatcher rd =  
    request.getRequestDispatcher("/profile.jsp");
```

Here:

- `request` → `HttpServletRequest` object
- `getRequestDispatcher()` → obtains the dispatcher
- `"/profile.jsp"` → destination resource
- `rd` → `RequestDispatcher` object

3.1 Methods of RequestDispatcher

`RequestDispatcher` mainly provides two important methods:

1. `forward()`
2. `include()`

4. forward() Method

Definition

`forward()` transfers the request from the current resource to another resource on the server.

The destination resource takes control of generating the response.

Syntax

```
rd.forward(request, response);
```

Example

```
RequestDispatcher rd =  
    request.getRequestDispatcher("/profile.jsp");  
  
rd.forward(request, response);
```

4.1 Characteristics of forward()

- It is a **server-side transfer**.
- The browser does **not make a new request**.
- The browser URL generally **remains unchanged**.
- The same request and response objects are passed to the destination.
- The destination resource generates the response.
- It is commonly used for Servlet → JSP communication.

4.2 Example: Login and Profile

Suppose a user successfully logs in.

The Login Servlet can forward the request to `profile.jsp`.

```
if (loginSuccessful) {  
  
    RequestDispatcher rd =  
        request.getRequestDispatcher("/profile.jsp");  
  
    rd.forward(request, response);  
}
```

Flow

```
Browser  
  |  
  | Request  
  v  
Login Servlet  
  |  
  | forward()  
  v  
Profile JSP  
  |  
  | Response  
  v  
Browser
```

Important

The request is transferred **inside the server**.

The browser does not know that the request was forwarded to another resource.

5. include() Method

Definition

`include()` includes the output of another resource in the current response.

Unlike `forward()`, the original resource can continue processing after the included resource finishes.

Syntax

```
rd.include(request, response);
```

Example

```
RequestDispatcher rd =  
    request.getRequestDispatcher("/index.html");  
  
rd.include(request, response);
```

5.1 How include() Works

```
Browser  
  |  
  | Request  
  v  
Servlet 1  
  |  
  | include()  
  v  
Servlet 2 / JSP  
  |  
  | Included Output  
  v  
Servlet 1  
  |  
  | Continue Processing  
  v  
Browser
```

Easy Understanding

```
include()
  ↓
Take output from another resource
  ↓
Add it to current response
  ↓
Original Servlet continues processing
```

5.2 Example: Login Error

Suppose a user enters an incorrect password.

The Servlet can:

1. Display an error message.
2. Include the login page.
3. Allow the user to try again.

Example:

```
response.setContentType("text/html");

out.println("<h3>Password did not match!</h3>");

RequestDispatcher rd =
    request.getRequestDispatcher("/index.html");

rd.include(request, response);
```

The resulting response can contain:

```
Password did not match!

+ Login Page
```

6. forward() vs include()

Feature	<code>forward()</code>	<code>include()</code>
Purpose	Transfers control to another resource	Includes another resource's output
Original resource	Stops handling the response after forwarding	Continues processing

Feature	<code>forward()</code>	<code>include()</code>
Response	Destination generates the main response	Included output becomes part of current response
Request	Same request	Same request
URL	Generally unchanged	Generally unchanged
Main idea	Transfer control	Add content

Remember

`forward()` = **Transfer control**

`include()` = **Include content and continue**

7. sendRedirect() vs RequestDispatcher

Feature	<code>sendRedirect()</code>	<code>RequestDispatcher</code>
Main purpose	Redirect browser to another URL	Handle resources internally
Processing	Browser/client-side redirection	Server-side handling
New request	Yes	No
URL	Changes	Generally unchanged
External URL	Can redirect	Mainly used for internal resources
Method/ Object	<code>HttpServletResponse</code>	Obtained from <code>HttpServletRequest</code>
Speed	Usually requires an additional round trip	Usually faster because no new browser request
Request attributes	Original request is not continued as the same request	Same request can carry attributes
Common use	Another URL or external website	Servlet → JSP / Servlet → Servlet

8. Difference in Request Processing

8.1 sendRedirect()

```

Browser
  |
  | Request 1
  v
Servlet 1
  |

```



```

    | sendRedirect()
    v
Browser
    |
    | Request 2
    v
Servlet 2 / New URL
    |
    | Response
    v
Browser

```

Key Point

`sendRedirect()` requires a new request from the browser.

8.2 RequestDispatcher forward()

```

Browser
    |
    | Request
    v
Servlet 1
    |
    | forward()
    v
Servlet 2 / JSP
    |
    | Response
    v
Browser

```

Key Point

`forward()` transfers the request internally without creating a new browser request.

8.3 RequestDispatcher include()

```

Browser
    |
    | Request
    v
Servlet 1
    |
    | include()
    v
Servlet 2 / JSP
    |

```

```
    | Output
    v
Servlet 1
    |
    | Continue
    v
Browser
```

Key Point

`include()` adds another resource's output to the current response.

9. Passing Data Using Request Attributes

One important feature of server-side request forwarding is that data can be stored as **request attributes**.

Definition

Request Attribute: Data stored in the current request so that another internal resource can access it.

Setting an Attribute

In the Servlet:

```
request.setAttribute("name_key", "Deepak");
```

Here:

- `"name_key"` → attribute name
- `"Deepak"` → attribute value

9.1 Forwarding the Request

```
request.setAttribute("name_key", "Deepak");

RequestDispatcher rd =
    request.getRequestDispatcher("/profile.jsp");

rd.forward(request, response);
```

Because the same request is forwarded, the destination resource can access the attribute.

9.2 Getting the Attribute in JSP

```
String myName =  
    (String) request.getAttribute("name_key");
```

The value of `myName` will be:

Deepak

Important for Exam

Request attributes are useful for passing data between resources using the same request.

10. Setting Content Type

When a Servlet generates HTML content, the response content type should be specified.

Syntax

```
response.setContentType("text/html");
```

This tells the browser that the response should be interpreted as HTML.

Example

```
response.setContentType("text/html");  
  
out.println("<h3>Password did not match!</h3>");
```

11. Practical Examples

Example 1: sendRedirect()

```
response.sendRedirect("https://www.google.com");
```

Meaning: The server tells the browser to go to Google.

Example 2: forward()

```
RequestDispatcher rd =  
    request.getRequestDispatcher("/profile.jsp");  
  
rd.forward(request, response);
```

Meaning: The request is internally transferred to `profile.jsp`.

Example 3: include()

```
RequestDispatcher rd =  
    request.getRequestDispatcher("/index.html");  
  
rd.include(request, response);
```

Meaning: The output of `index.html` is included in the current response.

Example 4: Passing Data

```
request.setAttribute("name_key", "Deepak");  
  
RequestDispatcher rd =  
    request.getRequestDispatcher("/profile.jsp");  
  
rd.forward(request, response);
```

In JSP:

```
String myName =  
    (String) request.getAttribute("name_key");
```

12. Commonly Confused Concepts

sendRedirect() vs forward()

sendRedirect()

- Browser is involved in the redirection.
- A new request is created.
- URL changes.
- Can redirect to an external URL.

forward()

- Server handles the transfer internally.
- Same request is used.
- URL generally remains unchanged.
- Used for internal resources.

Remember

`sendRedirect()` → Browser → NEW Request

`forward()` → Server → SAME Request

forward() vs include()

forward()

The current resource transfers control to another resource.

```
Current Resource
    ↓
  forward()
    ↓
Destination Resource
    ↓
Destination handles response
```

include()

The current resource includes another resource's output and continues processing.

```
Current Resource
    ↓
  include()
    ↓
Other Resource
    ↓
Output included
    ↓
Current Resource continues
```

13. Important Exam Points

Important for Exam:

- `sendRedirect()` is a method of the `HttpServletResponse` interface.

- `RequestDispatcher` is used for server-side communication between resources.
- `sendRedirect()` causes a **new request** from the browser.
- `forward()` does not create a new browser request.
- The browser URL **changes** with `sendRedirect()`.
- The browser URL generally **does not change** with `forward()` or `include()`.
- `forward()` transfers control to another resource.
- `include()` includes another resource's output in the current response.
- **Request attributes** can be used to pass data during internal request processing.
- `response.setContentType("text/html")` tells the browser that the response contains HTML.

14. Quick Revision Table

Concept	Easy Meaning
<code>sendRedirect()</code>	Tell the browser to go somewhere else
<code>RequestDispatcher</code>	Handle another resource internally
<code>forward()</code>	Transfer control
<code>include()</code>	Add another resource's output
<code>sendRedirect()</code>	New request
<code>forward()</code>	Same request
<code>include()</code>	Same request
<code>sendRedirect()</code>	URL changes
<code>forward()</code>	URL generally unchanged
<code>include()</code>	URL generally unchanged
<code>request.setAttribute()</code>	Store data in the request
<code>getAttribute()</code>	Retrieve stored request data

15. Easy Memory Trick

REDIRECT

↓

Browser gets redirected

↓

NEW Request

↓

URL Changes

FORWARD

↓

Server transfers control

↓

SAME Request

↓

URL stays the same

INCLUDE

↓

Include another resource's output

↓

Original resource continues

16. Key Points

- `sendRedirect()` redirects the browser to another URL.
- `sendRedirect()` creates a **new request**.
- The browser URL changes after `sendRedirect()`.
- `RequestDispatcher` is mainly used for internal server-side processing.
- `forward()` transfers control to another resource.
- `include()` includes another resource's output and allows the original resource to continue.
- `forward()` and `include()` use the **same request**.
- **Request attributes** can be used to pass data between internal resources.
- `sendRedirect()` is commonly used when the browser needs to go to another URL.
- `forward()` is commonly used for transferring control from a Servlet to a JSP.
- `include()` is useful when the output of another resource needs to become part of the current response.

17. Summary

`sendRedirect()`, `forward()`, and `include()` are important concepts in Servlet and JSP development.

`sendRedirect()` works through the browser. The server tells the browser to request another URL, so the browser creates a **new request** and the URL changes.

`RequestDispatcher` performs internal server-side processing. It provides two important methods:

- `forward()` → transfers control to another resource.
- `include()` → includes another resource's output while allowing the original resource to continue.

The most important difference to remember is:

`sendRedirect()` → NEW request + URL changes

`forward()` → SAME request + transfer control

`include()` → SAME request + include output

Final Memory Rule:

Redirect = New request

Forward = Transfer

Include = Add and continue